Rechnerarchitektur

Tutorium 1

Fynn (er/ihm)

Schreibt euren Rufnamen und Pronomen, wenn ihr möchtet (meine Pronomen sind z.B. er/ihm), auf das Klebeband.

Kennenlernspiel

Organisatorisches

Regelmäßige Teilnahme

- Teilnahme an n-2 Tutorien

Aktive Teilnahme

- mindestens 80% in n-1 online Quizzes
  - mindestens 80% in jedem einzelnen Quiz
  - Abgabe Freitag 23:55
- Bestehen von zwei Testaten über Assembly

Orga

- Email: fynn.voelkening@fu-berlin.de
- meldet euch jederzeit bei Fragen!

Was machen wir im Tutorium?

- Assemblyaufgaben im Gitlab: https://git.imp.fu-berlin.de/rechnerarchitektur/ra-exercises
- eigenes Tempo
- bei Fragen sagt Bescheid

KI-Nutzung

- überlegt euch, wie ihr KI lernförderlich einsetzen wollt
- arbeitet gemeinsam
- stellt Fragen im Tutorium

Assembly

NASM Assembly

- verschiedene Assembly Dialekte
- für Intel x86-64-Bit Linux-Kernel-Systeme

Sections

```
section .text
  ; hier steht euer Code

section .data
  ; hier stehen Variablen und Konstanten, die
  ; zu Beginn einen festen Wert haben

section .bss
  ; hier wird unitialisierter Speicherplatz reserviert
```

Direktiven und Funktionen

```
section .text
  global addition      ; macht die Funktion Addition für den
Linker sichtbar

  addition:
      ; hier kommt der Code von Addition hin
```

Instruktionen

```
section .text
    global foo

    foo:
        mov rdi, 2
        mov rax, 3
        mul rdi
        mov rsi, 3
        add rax, rsi
        ret
```

Register?

- kleine Speicherorte mit unterschiedlichen Funktionen (später mehr)
- ganzzahlige 64 Bit Register, die ihr erstmal benutzen dürft:
  - rax
  - rcx
  - rdx
  - rbx
  - rdi
  - rsi
  - r8-r11

mov

```
mov rax, 3    ; schreibt 3 in rax
mov rdi, rsi ; schreibt Inhalt von rsi in rdi
```

Addition und Subtraktion

```
add rax, 3   ; rax = rax + 3
add rdi, rsi ; rdi = rdi + rsi

sub rax, 3   ; rax = rax - 3
```

Multiplikation und Division

```
mul rdi         ; (rdx:rax) = rax * rdi

div rdi         ; teilt (rdx:rax) durch  rdi, Ergebnis in rax,
Rest in rdx
```

Return

```
ret             ; return Wert in rax
```

Mehr Operationen

https://github.com/Siedler/NASM-Assembly-Cheat-Sheet/blob/master/Cheat-Sheet.md

Was macht foo?

```
section .text
    global foo

    foo:
        mov rdi, 2
        mov rax, 3
        mul rdi
        mov rsi, 3
        add rax, rsi
        ret
```

Das ging alles etwas zu schnell?

- hier alles nochmal zum Nachlesen: https://git.imp.fu-berlin.de/rechnerarchitektur/ra-exercises/-/wikis/Assembly/Einf%C3%BChrung-NASM-Assembly

Was jetzt?

1. Main-branch vom git lesen und bearbeiten https://git.imp.fu-berlin.de/rechnerarchitektur/ra-exercises
2. Entwicklungsumgebung einrichten (nächste Slide)
3. Mit uebung-01 starten

Worauf kann ich arbeiten?

1. Poolrechner: anmelden und starten
2. Linux-Rechner:
   - installiert NASM
   - Debian: `sudo apt install nasm`
   - Fedora: `sudo dnf install nasm`
3. Windows mit WSL
   - WSL installieren, dann siehe Punkt 2
4. Andorra auf beliebigen Betriebssystem
   - Gebe im Terminal ein: `$ ssh ZEDAT_USER_NAME@andorra.imp.fu-berlin.de`

Womit soll ich starten?

```
section .text
  global addition

  addition:
      ; hier kommt euer Code rein
      ; Eingabeparameter sind in rdi und rsi
```